

LLM Integration Requirements Specification

Copyright (C)2025 Robin L. M. Cheung, MBA. All rights reserved.

Overview

This document specifies the requirements and architecture for Robinpedia's LLM integration, which goes beyond traditional Retrieval-Augmented Generation (RAG) to create a more intelligent, accurate, and contextually-aware knowledge system.

Core Requirements

1. Model Support and Provider Interfaces

Must Support

- **Ollama Local Models:** Primary focus for private, offline deployment
- **Provider Abstraction Layer:** Allow multiple LLM backends through a unified interface
- **Model Parameter Customization:** Temperature, top-p, context length, and other inference parameters
- **Stateful Conversations:** Maintain context across multiple exchanges
- **Streaming Responses:** Support incremental text generation

Nice to Support

- **Multi-model Orchestration:** Using different specialized models for different tasks
- **Provider Fallback Chains:** Graceful degradation when primary providers are unavailable
- **Quantization Level Selection:** Allow users to choose performance vs. quality tradeoffs
- **External API Providers:** OpenAI, Anthropic, etc. (with API key management)
- **Model Switching:** Dynamically change models mid-conversation

2. Knowledge Integration

Must Support

- **Structured Knowledge Graph Integration:** Connect LLM reasoning with the application's knowledge graph
- **ZIM Content Context Loading:** Dynamically include relevant article content in prompts
- **Answer Grounding:** Clearly connect responses to specific sources and evidence
- **Content Summarization:** Generate summaries of articles and sections

Nice to Support

- **Multi-hop Reasoning:** Connect information across multiple articles
- **Contextual Disambiguation:** Resolve entities and concepts based on user's reading history
- **Knowledge Graph Expansion:** Suggest new connections for the knowledge graph
- **Fact Verification:** Cross-check LLM outputs against knowledge base content

- **Source Triangulation:** Use multiple sources to validate information

3. Beyond Traditional RAG

Must Support

- **Semantic Entity Recognition:** Identify entities and concepts in both user queries and content
- **Query Decomposition:** Break complex queries into sub-problems
- **Confidence Scoring:** Report confidence levels for different parts of responses
- **Citation Generation:** Automatically generate proper citations for information
- **Hallucination Detection:** Flag when LLM may be generating unfounded content

Nice to Support

- **Active Retrieval:** LLM decides what content to retrieve and when
- **Information Synthesis:** Combine information from multiple sources coherently
- **Tool/Function Calling:** Allow models to request specific information or perform calculations
- **Bidirectional Knowledge Update:** Learn from user corrections and new information
- **Hypothetical Reasoning:** Explore "what if" scenarios based on existing knowledge

4. User Interaction and Experience

Must Support

- **Contextual Awareness:** Understand user's current article and reading context
- **Conversation History Management:** Save, load, and reference past conversations
- **Multi-turn Clarification:** Prompt for clarification on ambiguous queries
- **Explanation Generation:** Explain complex concepts at various detail levels
- **Annotation Integration:** Reference and incorporate user annotations in responses

Nice to Support

- **Personalized Knowledge Organization:** Adapt to user's knowledge level and interests
- **Learning Path Generation:** Suggest related articles to explore based on conversation
- **Proactive Information Offering:** Suggest relevant information without explicit queries
- **Natural Conversation Flows:** Support conversational patterns beyond QA exchanges
- **Multimodal Interactions:** Reference images, diagrams, and other visual elements

5. Performance and Technical Requirements

Must Support

- **Offline Operation:** Function with fully local models
- **Resource Efficiency:** Optimize for reasonable memory and CPU usage
- **Response Time Targets:** < 3 seconds for simple queries
- **Context Management:** Handle context windows efficiently
- **Error Handling:** Graceful degradation when models fail

Nice to Support

- **Hybrid Operation:** Combine local and remote models for optimal performance
- **Automatic Resource Scaling:** Adjust model size based on device capabilities
- **Inference Optimization:** Caching, batching, quantization for speed improvements
- **Low-resource Device Support:** Fallback modes for memory-constrained environments
- **Progressive Response Enhancement:** Show quick partial answers that improve with time

Architecture Components

1. Provider Layer

- **Provider Interface:** Abstraction for different LLM backends
- **Ollama Adapter:** Implementation for Ollama
- **Model Management:** Discovery, verification, and parameter control

2. Knowledge Integration Layer

- **Context Preparation:** Transform knowledge graph and content into LLM context
- **Retrieval Strategies:** Implementation of various retrieval approaches
- **Source Management:** Track and cite information sources

3. Reasoning Layer

- **Query Processing:** Parse, decompose, and refine user queries
- **Response Generation:** Combine retrieved knowledge with reasoning
- **Verification Engine:** Check factual accuracy and ground responses

4. Conversation Management

- **Session Handler:** Maintain conversation state and history
- **Context Window Optimization:** Efficiently manage limited context windows
- **Memory Management:** Decide what's important to remember across turns

5. UI Integration

- **Response Rendering:** Display LLM outputs with source highlighting
- **Interactive Elements:** Citations, expanding details, feedback mechanisms
- **Input Assistance:** Query suggestions, rephrasing help

Implementation Phases

Phase 1: Basic Ollama Integration

- Provider interface and Ollama adapter
- Simple context inclusion
- Basic conversation history
- Minimal UI for queries and responses

Phase 2: Enhanced Knowledge Integration

- Knowledge graph integration

- Citation and source tracking
- Query refinement
- Improved context selection

Phase 3: Advanced Reasoning

- Multi-hop reasoning
- Active retrieval
- Confidence scoring
- Tool/function calling

Phase 4: User Experience Enhancement

- Personalization
- Learning paths
- Multimodal integration
- Advanced UI components

Evaluation Criteria

1. **Accuracy:** Factual correctness of responses
2. **Relevance:** Appropriateness to user queries
3. **Context Awareness:** Understanding of user's situation
4. **Source Transparency:** Clarity about information sources
5. **Latency:** Response time under various conditions
6. **Robustness:** Handling of edge cases and errors
7. **Resource Usage:** Memory and CPU efficiency

Conclusion

This specification outlines a comprehensive approach to LLM integration that addresses the limitations of traditional RAG systems while providing a foundation for future capabilities. By focusing on knowledge grounding, context awareness, and flexible reasoning, Robinpedia aims to create a more accurate and useful knowledge exploration experience.

The implementation should prioritize correctness over completeness, starting with a solid foundation that can be expanded over time. This approach allows for adapting to the rapidly evolving LLM landscape while maintaining a consistent user experience.